IFN647 Week 3 review questions
SOLUTIONS

Q1 Which of the following descriptions is wrong? and justify your answer.

(1) The Boolean retrieval model is also known as exact-match retrieval since documents are retrieved if they exactly match the query specification, and otherwise are not retrieved.
(2) The process of developing queries with a focus on the size of the retrieved set has been called searching by numbers and is a consequence of the limitations of the Boolean retrieval model.
(3) In a vector space model, documents and queries are assumed to be part of a *t*-dimensional vector space, where *t* is the number of index terms (words, stems, phrases, etc.).
(4) There is an explicit definition of relevance in the vector space model.

Solutions: (4)

There is no explicit definition of relevance in the vector space model. There is an implicit assumption that relevance is related to the similarity of query and document vectors.

Q2 (Bayes Classifier)

Let Q = {US, ECONOM, ESPIONAG} be a query, and $C = \{D_1, D_2, D_3, D_4, D_5, D_6, D_7\}$ be a collection of documents. The following tables shows each document's terms and relevance to Q:

Document ID	Terms: d_{ij}	Relevance to Q
D_1	GERMAN, VW	0 no
D_2	US, US, ECONOM, SPY	1 yes
D_3	US, BILL, ECONOM, ESPIONAG	1 yes
D_4	US, ECONOM, ESPIONAG, BILL	1 yes
D_5	GERMAN, MAN, VW, ESPIONAG	0 no
D_6	GERMAN, GERMAN, MAN, VW, SPY	0 no
D_7	US, MAN, VW	0 no

Assume D = {US, ECONOM, ESPIONAG}, terms are independent, and a term's relevance probability is the term's occurs in relevant documents, calculate $P(D|R)$.

Solution:

Let T = {US, BILL, ECONOM, ESPIONAG, GERMAN, MAN, VW, SPY} = {d_1, d_2, d_3, d_4, d_5, d_6, d_7, d_8}, then D = {US, ECONOM, ESPIONAG} = {d_1, d_3, d_4}. So the binary vector of D is

(1, 0, 1, 1, 0, 0, 0, 0)

We also have three relevant documents D_2, D_3 and D_4 with the following binary vectors:

D_2 = (1, 0, 1, 0, 0, 0, 0, 1)
D_3 = (1, 1, 1, 1, 0, 0, 0, 0)
D_4 = (1, 1, 1, 1, 0, 0, 0, 0)

Let $p_i = P(d_i | R)$ be the probability that term d_i occurs (i.e., has value 1) in relevant documents, so we have

$p_1 = P(d_1 | R) = P(\text{'}US\text{'} | R) = 3/3 = 1.0$

$p_3 = P(d_3 | R) = P(\text{'ECONOM'} | R) = 3/3 = 1.0$

$p_4 = P(d_4 | R) = P(\text{'ESPIONAG'} | R) = 2/3 = 0.667$

By using the following equation:

$$P(D|R) = \prod_{i=1}^{t} P(d_i|R)$$

We have

$P(D|R) = P(d_1 | R) * P(d_3 | R) * P(d_4 | R) = 1 * 1 * 0.667 = 0.667$

Q3 (Probabilistic Models)

Let $N=|D|$ be the total number of documents in a training set D and R be the number of relevant documents in D. The following is a term weighting formula of probabilistic methods:

$$w_1(t) = \log[(r(t) / R) / (n(t) / N)]$$

Explain the meaning of $r(t)$ and $n(t)$ in the above formula.

Solution:

n(t) be the number of documents that contain term t; and
r(t) be the number of relevant documents that contain term t.

Q4 Which of the following is wrong? and justify your answer.

(1) A document feature is some attribute of the document we can express numerically.
(2) A ranking function takes data from document features combined with the query and produces a score.
(3) Topical features are the only features we can find in documents.
(4) If a document gets a high score, this means that the system thinks that document is a good match for the query, whereas lower numbers mean that the system thinks the document is a poor match for the query.

Solution: (3)
A document also has Quality features (e.g., meta information, such as days since last update of the document).

Q5 (Abstract Model of Ranking)

Assume the model of ranking contains a ranking function $R(Q, D)$, which compares each document with the query and computes a score. Those scores are then used to determine the final ranked list.

An alternate ranking model might contain a different kind of ranking function, $f(A, B, Q)$, where A and B are two different documents in the collection and Q is the query. When A should be ranked higher than B, $f(A, B, Q)$ evaluates to 1. When A should be ranked below B, $f(A, B, Q)$ evaluates to –1. If you have a ranking function $R(Q, D)$, show how you can use it in a system that requires one of the form $f(A, B, Q)$.

Solution:

For two documents A and B, we can define $f(A, B, Q) = 1$ if $R(Q, A) > R(Q, B)$; otherwise, if $R(Q, A) < R(Q, B)$, $f(A, B, Q) = -1$.

$$f(A,B,Q) = \begin{cases} 1, & if\ R(Q,A) > R(Q,B) \\ -1, & if\ R(Q,B) > R(Q,A) \end{cases}$$

For the special case of $R(Q, A) = R(Q, B)$, we may extend the definition to let $f(A, B, Q) = 0$, or assign 1 or -1 to $f(A, B, Q)$.

Q6 (Word Embedding)

Word2Vec can be used to generate distributed word vector representations in Python. The required modules are NLTK and Gensim. Install them on your computer, then import the necessary modules.

In this task, you will write a Python program that reads a text file (alice.txt), tokenizes the text into sentences using sent_tokenize(), and further tokenizes each sentence into words using word_tokenize(). The processed data should be stored as a list of lists, where each inner list contains the words of a single sentence.

Next, you will train a CBOW or Skip-gram Word2Vec model using gensim.models.Word2Vec. Finally, you will use the model's wv.similarity() method to compute:

(1) The semantic similarity between **"alice"** and **"wonderland"**
(2) The semantic similarity between **"alice"** and **"machines"**

Solution:

```
# importing all necessary modules
from nltk.tokenize import sent_tokenize, word_tokenize
import gensim
from gensim.models import Word2Vec
```

```
#  Reads 'alice.txt' file
sample = open("alice.txt") # sample = open("alice.txt", "utf8")
s = sample.read()

# Replace escape character with space
f = s.replace("\n", " ")
data = []

# Iterate through each sentence in the file
for i in sent_tokenize(f):
    temp = []
    # tokenize the sentence into words
    for j in word_tokenize(i):
        temp.append(j.lower())
    data.append(temp)
```

```
# Create CBOW model
model1 = gensim.models.Word2Vec(data, min_count = 1, vector_size = 10, window = 5)

# Print results
print("vector of alice = ", model1.wv['alice'])
print("vector of wonderland = ", model1.wv['wonderland'])
print("vector of machines = ", model1.wv['machines'])
print("Cosine similarity between 'alice' " + "and 'wonderland' - CBOW : ", model1.wv.similarity('alice', 'wonderland'))
print("Cosine similarity between 'alice' " + "and 'machines' - CBOW : ", model1.wv.similarity('alice', 'machines'))

# you may test another model, Skip gram model
# model2 = gensim.models.Word2Vec(data, min_count = 1, vector_size = 50, window = 6, sg = 1)
```

```
vector of alice =  [-0.61964875 -0.6991252   1.3847291   0.4731636  -0.07328504  0.5592312
  2.3514514   1.9336985  -1.9396105  -0.6048208 ]
vector of wonderland =  [-0.02551305 -0.12024416  0.22096843  0.11155266  0.04218432  0.21726419
  0.2830255   0.32678026 -0.40621445 -0.01513698]
vector of machines =  [-0.02524116 -0.06399771  0.09318245 -0.04946016  0.08125422 -0.02004074
  0.04810669  0.04457665 -0.08520888  0.08923841]
Cosine similarity between 'alice' and 'wonderland' - CBOW :  0.94524586
Cosine similarity between 'alice' and 'machines' - CBOW :  0.545615
```

Q7 (Optional) Word Embedding: BERT

BERT word embeddings are contextualized vector representations of tokens produced by BERT's Transformer encoder. The following code demonstrates how to use a standard pretrained BERT base model to encode two input texts (`text1` and `text2`) and generate their corresponding token-level (word) embeddings.

```
# Installing transformers and torch modules if your computer does not insatll them
# !pip install transformers
# !pip3 install torch torchvision
```

```
# importing libraries
import random
import torch
from transformers import BertTokenizer, BertModel
from sklearn.metrics.pairwise import cosine_similarity
```

```
# Set a random seed
random_seed = 42
random.seed(random_seed)
# Set a random seed for PyTorch (for GPU as well)
torch.manual_seed(random_seed)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(random_seed)
```

```
# Load BERT tokenizer and model, where
#'bert-base-uncased' is one of the standard pretrained BERT base models released by Google:
## text is lowercased, Hidden size 768, about 110M parameters
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')
```

```
# Input texts
text1 = "Machinelearning for Natural Language Processing"
text2 = "Machinelearning for text data analysis"
# Tokenize and encode text using batch_encode_plus
# The function returns a dictionary containing the token IDs and attention masks
encoding = tokenizer.batch_encode_plus( [text1, text2],# List of input texts
    padding=True,              # Pad to the maximum sequence length
    truncation=True,           # Truncate to the maximum sequence length if necessary
    return_tensors='pt',      # Return PyTorch tensors
    add_special_tokens=True    # Add special tokens CLS and SEP
)
```

```
# Generate embeddings using BERT model
input_ids = encoding['input_ids']  # Token IDs
attention_mask = encoding['attention_mask']  # Attention mask
with torch.no_grad():
    outputs = model(input_ids, attention_mask=attention_mask)
    word_embeddings = outputs.last_hidden_state  # This contains the embeddings

print("Shape of Word Embeddings:")
print(word_embeddings.shape)
#The shape of Word embeddings is [1, 12, 768] where 768 is the dimensionality/hidden size of the word embeddings, i.e.,
# Each token is represented by a 768-dimensional vector and 10 is the number of tokens in our input text after tokenization.
# Here 2 is the total number of sentences we have passed.
```

```
Shape of Word Embeddings:
torch.Size([2, 10, 768])
```

(1) Compute the average of word embeddings to get sentence embeddings.
(2) Compute the cosine similarity between text1 and text2.

Solution:

(1)

```
# Compute the average of word embeddings to get the sentence embedding
sentence_embedding = word_embeddings.mean(dim=1)  # Average pooling along the sequence length dimension

# Output the shape of the sentence embedding
print("Shape of Sentence Embedding:")
print(sentence_embedding.shape)
```

```
Shape of Sentence Embedding:
torch.Size([2, 768])
```

(2)

```
# Compute cosine similarity between text1 and text2
sent1_emb = sentence_embedding[0]
sent2_emb = sentence_embedding[1]
similarity_score = cosine_similarity(sent1_emb.unsqueeze(0), sent2_emb.unsqueeze(0))
# Print the similarity score
print("Cosine Similarity Score:")
print(similarity_score[0][0])
```

```
Cosine Similarity Score:
0.9200219
```